Generative AI: Video 2 Notes

LLMs, Transformers, Tokens and Tokenization

Series: Complete GenAI series with Python and LangChain

Note: some words in the auto-generated transcript were garbled. Corrected from context: "Lunchan" = LangChain, "Google Colum" = Google Colab, "tick token" = tiktoken, "climb" (in the bank example) = "claim" in the transcript, "We finally sat" = "be finally set" in the transcript.

1. Topics Covered in This Video

- What is an **LLM** and how does it work?
- What are **tokens** and **tokenization**?
- Implementing tokens and token IDs in **Python** using a library (done in Google Colab).
- Bonus concepts: **context window** and **token limits (max tokens)**, including how pricing works.

2. What is an LLM?

LLM = Large Language Model

Word	Meaning
Large	Trained on a huge amount of data
Language	Human language, i.e. text content
Model	An artificial machine, essentially a mathematical function

An LLM is an artificial machine trained on huge amounts of **text data**. Examples: ChatGPT, Gemini, Claude.

3. GPT: Generative Pre-trained Transformer

"ChatGPT" means chatting with GPT. The full form of GPT explains how all LLMs work.

3.1 Generative

- The ability to generate new content based on the user's input.

3.2 Pre-trained

- LLMs are trained beforehand on huge amounts of text data: **books, articles, websites, programming languages**, and practically all data available on the internet.
- When a user asks a question, the LLM generates new content based on this pre-trained knowledge.

3.3 Transformer

- A **special type of neural network architecture** that can understand the user's text input deeply.
- Based on that understanding and its pre-trained data, it generates new text.
- Introduced by **Google in 2017** in the research paper **"Attention Is All You Need"**.
- The key idea introduced in that paper is the **Attention Mechanism**.

Attention Mechanism

It lets the LLM establish **relationships between the different words** in the input.

Example: "The bank was too steep to climb."

- The word **bank** could mean a **financial bank** or a **river bank**.
- The attention mechanism also looks at the other words in the sentence (such as **steep** and **climb**).
- Because of this relationship, the transformer understands that the sentence is about a **river bank**, not a financial one.

4. What Do Transformers Actually Do?

A transformer predicts the next word based on the input, then repeats the process.

Example walkthrough:

Step	Input to the transformer	Predicted next word
1	We finally sat	down
2	We finally sat down	on
3	We finally sat down on	the
4	We finally sat down on the	bench

- Each predicted word is **appended to the input**, and the new input is used to predict the next word.
- At each step the model predicts **multiple possible words** (for example bench, chair, floor), each with a probability.
- The word with the **highest probability** is chosen.
- When the model decides nothing more needs to be predicted, the final answer is shown on the UI.
- **All LLMs work the same way.** Gemini, Claude and Google's other models are also transformer-based and pre-trained on data, and they generate text like GPT does.

5. Tokens and Tokenization

Why are tokens needed? Computers work on binary and can never directly understand text. So text must be converted into numbers before an LLM can process it.

The Process, Step by Step

Step	What happens	Example: "Who is PM of India?"
1. Chunking	The input text is broken into smaller pieces (chunks).	Who / is / PM / of / India / ?
2. Tokens	The pieces form a list of strings. These are called **tokens**.	A list of 7 tokens
3. Token IDs	Each token is looked up in the model's dictionary (vocabulary) and given a **unique number**.	A list of 7 numbers
4. LLM processing	The token IDs are sent to the LLM. It understands the numbers and outputs a new set of numbers.	Output IDs

Step	What happens	Example: "Who is PM of India?"
5. Decoding	The output IDs are mapped back to words using the same dictionary, combined, and shown to the user.	The final answer text

Term	Definition
Token	A piece of text obtained after breaking the input (a chunk).
Token ID	The unique number assigned to a token from the model's dictionary.
Tokenization	The whole process of converting text into tokens (and token IDs).
Tokenizer	The component that converts the input string into tokens.

Important: every LLM has its **own individual dictionary** (mapping of each unique text piece to a number). So the same text gets **different token IDs in different models**.

6. Tokenizer Visualization (Demo)

- Tools used: the **Hugging Face** tokenizer playground and the **OpenAI** tokenizer (search "tokenizer visualization").
- Input: **"Who is PM of India?"** gave **7 tokens**: "Who", " is" (with the leading space), " P", "M", " of", " India", "?".
- The **Token IDs** view shows 7 numbers as a list.
- In the OpenAI tokenizer, switching the model (GPT-4o, GPT-3.5, GPT-3, GPT-4) **changes the token IDs**. This confirms each model has its own dictionary.
- Platforms like OpenAI do this behind the scenes in every request.

7. Coding Tokenization in Python (Google Colab)

The instructor used **Google Colab** (a cloud notebook) only for this video. From the next videos, everything will be set up on the local machine.

Colab Setup

- Search "Google Colab", open it and choose **New Notebook**. Rename it (for example "Tokens and Token IDs").
- Click **Connect** (RAM and disk get allocated).
- Test with a simple cell such as **5 + 5**, which gives **10**. Run a cell with the play button, or Shift+Enter (Ctrl+Enter also runs the cell).

Encode: text to token IDs

```
import tiktoken

text = "Who is PM of India?"

# Create the tokenizer for a specific model
tokenizer = tiktoken.encoding_for_model("gpt-4")

token_ids = tokenizer.encode(text)
print(token_ids)   # 7 token IDs
```

- The library used is **tiktoken**, a popular tokenizer library.
- The model name must be passed, because each model has its own dictionary.
- The output had **7 IDs**, exactly the same as the Hugging Face tokenizer tool (the instructor compared them).

Decode: token IDs back to text

```
# IDs copied from the tokenizer tool for "PM of India is Modi ji"
result_ids = [ ...ids from the tool... ]

result = tokenizer.decode(result_ids)
print(result)   # PM of India is Modi ji
```

- Decoding returned the text **exactly** as before, with no change in case or spacing.
- This mirrors what happens in real use: the LLM returns token IDs, and the tokenizer decodes them into readable text.

8. Context Window

- **Definition:** how much data (in tokens) can be sent into an LLM at once and how much can be received from it at once.
- **Analogy:** a house has a front gate (input) and a back gate (output). Each can be a different size, which decides how many people can enter or leave at once. Likewise, the context window is **how many tokens can go in and come out**.
- **Real-life example:** older ChatGPT models often showed a "memory is full" warning after many questions and answers. The conversation had already used up the model's context window, so no more data could be sent or received.

9. Token Limits (Max Tokens) and Pricing

- When working with LLMs as a GenAI developer, you decide the **maximum tokens** to send and to receive in each call.
- **Why it matters:** paid LLM providers (OpenAI, Google Gemini, Claude and others) **charge based on tokens**.
- **Input tokens are charged**, **output tokens are charged**, and even **cached input** (data stored in memory) is charged.
- **Example from OpenAI's pricing page (as shown in the video):** about **$1.25 per 1 million input tokens** and **$10 per 1 million output tokens**. Prices differ by model, so check the current pricing page.
- Setting **max tokens** yourself gives you **control over cost**. This will be used in the upcoming videos.

Quick Revision Summary

- **LLM** = Large Language Model, an AI model trained on huge amounts of text.
- **GPT** = Generative Pre-trained Transformer. Generative means it creates content, pre-trained means it learned from huge data beforehand, and the transformer is the architecture.
- **Transformer** (Google, 2017, "Attention Is All You Need") uses the **attention mechanism** to relate words to each other.
- LLMs work by **predicting the next word** repeatedly, choosing the highest-probability word each time.
- Text becomes **tokens**, tokens become **token IDs** (via the model's dictionary), and the LLM works on the numbers. This is **tokenization**.
- Different models have different token IDs for the same text.
- **tiktoken**: `encode()` gives IDs, `decode()` gives text.
- **Context window** = how many tokens can go in and come out at once.
- **Max tokens** controls how many tokens you send and receive, which controls cost.

What's Next

From the next video, the series moves to **practical work**: writing code and building GenAI applications, with an interesting project for each topic.